

Curso de especialización en Desarrollo de aplicaciones en lenguaje Python

UT4.- Acceso a bases de datos con Python

Tarea 3

1. Los ejercicios que siguen están pensados para trabajar con el **ORM de Django**. Por tanto para realizarlos debes:
 - Crear un proyecto Django al que llamaremos **Empleados ORM**.
 - Crear dentro del proyecto una aplicación llamada empleados.
 - Registrar la app en el archivo **settings.py**.
 - La base de datos. Trabajaremos con SQLite y la base de datos la llamaremos **empleados.db**.
 - Abrir el archivo models.py y crear el modelo Empleado. Estudia la imagen siguiente.

```
from django.db import models
# Create your models here.
class Empleado(models.Model):
    nombre = models.CharField(max_length=50)
    apellidos = models.CharField(max_length=50)
    salario = models.DecimalField(max_digits=8, decimal_places=2)
    departamento = models.TextField(max_length=50, blank=True, null=True)
    fecha_contratacion = models.DateField(null=True, blank=True, auto_now=True)

    def __str__(self):
        return f"{self.nombre} {self.apellidos} - {self.salario} - {self.departamento} - {self.fecha_contratacion}"
```

- Haz las migraciones para crear la tabla en la base de datos.
- Diseña los siguientes scripts. Para que estos scripts sean funcionales deben incluir al principio de los mismos las siguientes líneas.

```
import os
import django

# Configuración del entorno Django (ajusta con tu proyecto)
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'Empleados ORM.settings')
django.setup()
```

- Script 1. Insertar varios empleados.
- Script 2. Listar los empleados.
- Script 3. Listar los empleados del departamento de Ventas.
- Script 4. Buscar un empleado por su id.
- Script 5. Listar los empleados ordenados por apellidos y nombre.
- Script 6. Listar los empleados que no pertenecen al departamento de Ventas.
- Script 7. Listar los empleados con la cadena 'cía' en el atributo apellidos.
- Script 8. Listar los empleados del departamento de Ventas con salario superior a 2000 euros.

- Script 9. Listar los tres empleados con mayor salario.
 - Script 10. Eliminar un empleado localizado por su id.
 - Script 11. Actualizar el salario de un empleado localizado por su id.
 - Script 12. Calcular salario mínimo, máximo, medio y contar los empleados del departamento de ventas.
2. Seguimos trabajando con los modelos de Django. Crearemos ahora una nueva aplicación Django con las siguientes características:
- Nombre del proyecto: **CursosFormacion**.
 - Nombre de la app: **cursos**.
 - Base de datos **mysql**: cursos.
 - Modelo.

```
class Curso(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField()
    horas = models.IntegerField()
    tipo = models.CharField(max_length=50)
    fecha_inicio = models.DateField()
    fecha_fin = models.DateField()
    precio = models.DecimalField(max_digits=10, decimal_places=2)

    def __str__(self):
        return f"Curso: {self.nombre} - descripcion: {self.descripcion} - horas: {self.horas} - tipo: {self.tipo} - " \
            f"fecha inicio: {self.fecha_inicio} - fecha fin: {self.fecha_fin} - precio: {self.precio}"

class Meta:
    db_table = 'cursos'
```

- Scripts.
 1. Insertar varios cursos en la base de datos.
 2. Mostrar todos los cursos "online"
 3. Obtener todos los tipos de cursos.
 4. Contar los cursos por tipos.
 5. Mostrar los que no están iniciados.
 6. Mostrar los cursos que tienen una duración entre dos valores.
 7. Actualizar el precio de todos los cursos presenciales, sumarle 50 euros a cada uno.
 8. Cursos que están desarrollándose.
 9. Crear un PDF con los cursos próximos. Aquellos que no han comenzado

3. Crear el siguiente proyecto Django y los scripts asociados.

- Nombre del proyecto: GestionTareas.
- Nombre de la app: tareas.
- Base de datos **mysql**: [tareas](#).
- Modelo

```
# Create your models here.
class Tarea(models.Model):
    titulo = models.CharField(max_length=100)
    descripcion = models.TextField()
    fecha_creacion = models.DateTimeField(null=True, blank=True)
    fecha_finalizacion = models.DateTimeField(null=True, blank=True)
    completada = models.BooleanField(default=False)

    def __str__(self):
        return self.titulo

    class Meta:
        db_table = "tareas"
```

- Scripts.
 1. Insertar varias tareas.
 2. Mostrar las completadas.
 3. Mostrar las tareas de un año.
 4. Contar las tareas por año.
 5. Actualizar las tareas de un año a completadas.
 6. De las tareas completadas, mostrar la diferencia de horas entre su creación y finalización.